

FACSIMILIA

Game design: how the world works

This is the design plan for the game: every system, what it does, and the reasoning behind it. Each chapter opens with a short summary in plain English; the detail after it is for when you want the full picture.

(DECIDED) settled and either built or ready to build

(PROPOSED) drafted, waiting for your approval

(OPEN) not decided yet, or a value to tune later

Made from MECHANICS.md on 25 September 2026.

CONTENTS

DESIGN DECISIONS

CORE INTERACTION PRINCIPLE: THE BRUSH STAYS AUTHORITATIVE

LAYERS

Invariant: ownership on organized land lives on the province, not the cell
Provinces as built

UNCLAIMED VS. UNORGANIZED LAND — TWO DIFFERENT THINGS

Settlement naming threshold — percentile rank, with hysteresis

POPULATION & SETTLEMENT ENGINE

1. Historical gravity: the attractor field
 2. Heatmap-to-population translation
- Historical ceiling: only the player exceeds history
Capital growth bonuses
Starting and seed populations
Growth drivers: what makes people, and what moves them
3. Settlement naming and spawning
- Geographic regions — the ancient geographers' map

RESOURCES AND THE LAND

Three tiers, not one list
The land layer
Each crop and herd its own profile
Change over time: four speeds
History as the guide, here too
Goods list: additions and grouping
What it feeds
Suggested build order within step 2

DYNASTIES AND CHARACTERS

NEW CONCEPTS NEEDED

THE FIVE SIMULATION SYSTEMS

MIGHT: THE UNIFIED STRENGTH SCORE

ANNEXATION UX: FOG OF WAR AND THE FLOATING CONFIRMATION PANEL

ANNEXATION RESOLUTION

SUGGESTED BUILD ORDER

DESIGN DECISIONS

(DECIDED, 25 SEP 2026)

IN SHORT

Your answers to the big design questions: sandbox with optional goals, realms that make their own choices but lean on history, full war and diplomacy, historical new realms plus revolts, culture and religion, player-chosen pacing for long games, tribes that start unorganized, and personal unions guided by history.

Answered by the designer in the Facsimilia Ledger. History is the guiding factor throughout: where these leave room, follow what really happened unless the player is involved.

- **Goal: sandbox with optional goals.** The game never has to end. It tracks a score and offers goals (hold a region, outlast rivals, rebuild Alexander's empire...) that the player may chase.
- **Other realms: their own choices, with history as a pull.** Realms decide for themselves but lean toward what they historically did, more strongly where the player isn't involved. (This matches the population engine's design: bots follow history, the player can bend it.)
- **War and diplomacy: full.** War must be declared before painting into a realm; peace deals end it; alliances and vassals exist. Conquest by painting (build step 7) works inside a declared war.
- **New realms: historical arrivals, revolts and civil wars.** New realms appear at their real dates and places (Galatia 278 BC, Parthia 247 BC, ...). Revolts and civil wars follow history too, *unless* they happen in the player's lands or the player causes them; there the simulation decides.
- **Culture and religion: both,** with conversion and assimilation over time; together they drive integration (build step 6). Characters already carry naming cultures.
- **Pacing: the player chooses years per turn.** Games are meant to be long: a playthrough taking weeks or months is the target, not a problem. Design for depth over speed.
- **Frontier tribes start unorganized** (built): the Iberian, Gallic and Scythian peoples were confederations, not administered states, so their land starts with no provinces and follows the unorganized-land rules.
- **Personal unions: allowed, guided by history.** One ruler may inherit a second throne. The realms stay separate unless history merged them; if history renamed the realm on inheritance, the game does too. A union only splits again when the player is involved or history split it. (Not built yet: until it is, a monarch is still passed over for a second throne.)
- **Growth drivers: approved as drafted** (see Growth drivers).

CORE INTERACTION PRINCIPLE: THE BRUSH STAYS AUTHORITATIVE

IN SHORT

You shape the world with one tool: a round brush. Conquest, provinces and anything added later all use that same brush, instead of menus or separate editors.

The existing left-drag-to-paint / right-click-to-clear ownership brush (`src/World/MapView.cs`, `PaintRadius`, the proposal overlay) is the one mechanic everything else builds on top of, not around. Every new layer described below reuses that same brush code path (radius stamp, drag-paint, dirty-cell tracking) rather than introducing a different interaction paradigm (no click-to-select-a-region UI, no modal province editor). New layers just give the brush a different target to paint onto.

LAYERS

IN SHORT

The map is stored as stacked layers over the same 45-million-cell grid: who owns each spot, which province it belongs to, and how many people and resources are there. A province's numbers are always added up from its cells, so redrawing a border can never break anything. "Provinces as built" describes what is in the game today.

Three per-cell layers, all the same 8192x5476 shape as the existing ownership grid:

1. **Ownership** (`OwnershipGrid.cells`, existing) — which realm holds this cell. Painted directly today (immediately, no commit step yet).
2. **province_id** (new) — which province this cell administratively belongs to. `0` = unorganized. Brush-editable in a "province mode" that targets this layer instead of ownership, with a selected target (an existing province of yours, to expand it, or "new province", to found one).
3. **Density fields** (new) — population and raw-resource presence. Population is **live, not baked**: it's $C * H_{final}^k$, recomputed each population tick from the blended simulated and historical heatmaps (see Population & settlement engine, below). HYDE (see Might, below) provides the historical side of that blend and seeds the 300 BC starting state. Raw-resource presence is still baked at world-gen from terrain data already loaded for the terrain shader. Not normally player-painted (geography/history-driven), though the same brush code could hand-adjust them in a dev/authoring context.

Provinces never store population/resources as an opaque blob — a province's stats are always the sum of the density fields over whatever cells currently carry its `province_id`. This is what makes provinces safely brush-editable: resizing a province's boundary just changes what it sums over, nothing needs manual splitting/merging logic. (DECIDED)

INVARIANT: OWNERSHIP ON ORGANIZED LAND LIVES ON THE PROVINCE, NOT THE CELL

For a province (`province_id != 0`), the *province* is the authoritative owner (`realm_of_province[province_id]`) — every member cell is owned by whoever owns the province, always wholly, never partially. `OwnershipGrid.cells` still holds a per-cell realm id (unchanged, so the renderer and existing tests keep reading it exactly as cheaply as today), but for organized cells it's a write-through cache: whenever a province's owning realm changes, every one of its member cells gets bulk-written to match in one pass — the same bulk-write pattern `_seed_real_civs` already uses for the political mask — never decided cell-by-cell.

For unorganized land (`province_id == 0`), there's no province to defer to: `OwnershipGrid.cells` is the sole, direct source of truth, exactly as it works today. Ownership there still changes cell-by-cell via the brush, independent of any province. (DECIDED)

PROVINCES AS BUILT (DECIDED, BUILT — BUILD STEP 1)

Code: `src/World/Provinces.cs` (the layer and records), `src/World/ProvinceGenerator.cs` (the 300 BC provinces), `src/World/MapView.Provinces.cs` / `MapView.ProvinceBrush.cs` (drawing, selection, brush), `src/UI/Hud.Provinces.cs` (panel and map modes). Test: `ProvinceTest`.

- **Layer.** One province id per cell (`ushort`, 0 = unorganized), saved as `provinces.bin` beside the grid; names and owners go in `state.json`. Stats are always summed over cells (area now; population from the engine).
- **The 300 BC provinces.** `data/province_seeds.json` lists 245 historical regions c. 300 BC (Latium, Campania, Attica, Babylonia, Media, the Egyptian Delta, Meroe, Celtiberia, Arvernia, Taurica...). Each one on a realm's land becomes a province of that realm; provinces grow from their seeds over the realm's own land until they meet, then their shared borders get a gentle meander. Wherever land is more than ~150 km from any region, an extra province is added and named from the nearest region ("Northern Media"). Unseeded islands under ~1,500 km² join the nearest province. Result: two-thirds of them historical names, median ~21,500 km², about 190 provinces in all. The frontier tribes (Iberia, Gaul, Scythia) start unorganized, with no provinces (see Design decisions).
- **On the map.** Province borders are thin and fainter than realm borders, getting stronger as you zoom in; province names appear from about 2x zoom. Unorganized owned land shows paler. The selected province is lightened and outlined in gold.
- **Map modes** (HUD buttons or keys 1-3): *Inspect* (click selects a province, drag pans), *Provinces* (the province brush), *Conquest* (the old proposal brush, not yet playable).
- **Province brush.** Same round brush as conquest, a constant size on screen. Left-drag adds the player's own land under it to the selected province (one of the player's); right-drag takes land out of any province, leaving it unorganized. It never touches other realms' land. *New Province* finds an empty one to paint into; a province whose last cell is painted away disappears. The player can rename their own provinces in the panel.
- **Invariant.** `ProvinceMap.SetRealm` hands a whole province to another realm and rewrites every member cell's ownership in one pass.

UNCLAIMED VS. UNORGANIZED LAND — TWO DIFFERENT THINGS

IN SHORT

Unclaimed land has no owner at all. Unorganized land has an owner but no province yet, so it is weaker: no taxes, only dirt roads, just a local militia. That weakness is the reason to organize land you take.

Easy to conflate, so pinning it down: **unclaimed** land (`owner_id == 0` in the existing ownership grid — today's "wild" cells) has no owning realm at all. **Unorganized** land has a realm owner, just no province yet (`province_id == 0`). They share one mechanic and diverge on everything that depends on having an owner to credit output to.

Population is a base geographic fact, independent of ownership — the density field (see Layers, above) exists on *every* land cell, unclaimed or not. Unclaimed land has its own (typically sparse) native population, same as unorganized or organized land does. Local militia strength derives from that population everywhere it applies: unclaimed land resists incursion the same way unorganized land resists conquest, scaled by however populated it actually is. What differs is output — unclaimed land has no owning realm to credit raw resources to, so **it produces nothing for no one**, even though the same coal/ore/water/flora deposits are physically present in its density field. The moment it's claimed and becomes unorganized territory of some realm, that realm starts drawing the base-rate raw output described below. This also refines the earlier assumption that settling unclaimed land is an unconditional freebie: it's cheap in practice (native population, and therefore militia, is usually sparse) but not literally free — it still resolves against whatever local defense exists. **(DECIDED)**

Newly gained land (settled or conquered) defaults to `province_id = 0`. It's real territory, but capped well below an organized province:

CAPABILITY	UNCLAIMED (NO OWNER)	UNORGANIZED (OWNED, NO PROVINCE)	ORGANIZED (IN A PROVINCE)
Population	Yes — sparse, native	Yes	Yes
Raw resource extraction (coal, ores, water, flora/fauna)	None — no owner to credit it to	Yes, base rate, credited to owner	Yes, improvable with infrastructure
Resource processing	No	No	Yes
Roads	None built; at most a natural/trade trail passing through	Dirt & gravel only	Full tiers as tech allows
Electricity routes	Cannot pass through	Cannot pass through	Allowed once unlocked

CAPABILITY	UNCLAIMED (NO OWNER)	UNORGANIZED (OWNED, NO PROVINCE)	ORGANIZED (IN A PROVINCE)
Military / defense	Population-scaled local militia, resists incursion	Population-scaled local militia, non-deployable	Full army stationing + logistics
Taxation	None (no owner)	None	Yes
Named settlements	Suppressed, same highly-populous exception applies	Suppressed, same highly-populous exception applies	Full, plus founding new ones

Deliberately weak defense on both unclaimed and unorganized land is the incentive to actually organize conquered/settled territory rather than leave it as paperwork. (DECIDED)

SETTLEMENT NAMING THRESHOLD — PERCENTILE RANK, WITH HYSTERESIS

Supersedes the earlier "10% of the world's largest settlement" rule (and the flat 50,000 placeholder before that). Both of those measured *size*; what the rule actually needs to protect is *map readability*, and that's a question of how many labels are on screen, which a percentile controls directly. Full rules, including where the populations being ranked come from, are in **Population & settlement engine**, below. Short version: a population cluster gets named when it enters the top **2-5%** of active clusters, and keeps its name until it falls out of the top **20%** or hits zero. This is the "highly populous exception" the capability matrix above refers to. It applies the same way on unclaimed, unorganized, and organized land.

(DECIDED)

POPULATION & SETTLEMENT ENGINE

Three stages, run in this order every population tick:

1. **Blend** the simulated and historical heatmaps into one `H_final`.
2. **Translate** `H_final` into a target population per node, and move each node's actual population a limited step toward it.
3. **Rank** clusters by population, and name or un-name settlements.

I. HISTORICAL GRAVITY: THE ATTRACTOR FIELD

Two heatmaps are kept, both normalized to 0.0-1.0 per node:

- `H_sim` (simulated heat) is driven by gameplay: food, water, trade routes, infrastructure the player or AI builds, war damage.
- `H_hist` (historical target heat) is a static, invisible reference, never shown to the player. It carries e.g. a heavy spike over Rome or the Nile Delta in antiquity.

$$H_{\text{final}} = (1 - \alpha) * H_{\text{sim}} + \alpha * H_{\text{hist}} \quad \alpha \in [0.15, 0.20]$$

A convex blend of two 0-1 fields stays in 0-1, so no renormalization is needed. Since $H_{\text{final}} \geq (1 - \alpha) * H_{\text{sim}}$, a player who builds heavily in a historically barren region always keeps at least 80-85% of their own heat there. `H_sim` overpowers the historical pull, which is what makes alternate history possible.

(DECIDED)

Source of `H_hist`: HYDE, not a hand-authored map. (decided) HYDE is already the decided source of the population density field (see Might, below), and it ships gridded population snapshots from 10,000 BCE to the present (the game uses HYDE 3.2.1, which ends at 2017). That's exactly a sequence of historical target maps, baked onto this projection by `build_population_mask.py`. So `H_hist(year)` = the HYDE snapshot for that year, normalized to 0-1. Between HYDE's snapshots (per-century from 0 AD to 1700, per-decade to 2000, yearly after), interpolate linearly so the pull moves smoothly instead of jumping at each keyframe. HYDE has no 300 BC snapshot: before 0 AD it steps by 1000 years. The 300 BC keyframe is derived from 1000 BC and 0 AD per node, assuming steady exponential growth in between (39.4 million people on the map, against 26.1 million at 1000 BC and 48.2 million at 0 AD). At the 300 BC start, `H_sim` is seeded from `H_hist` itself, so turn one is historical by construction and divergence only comes from play.

The slow pull comes from population feedback into `H_sim`. (decided) On its own the blend is instantaneous and memoryless. An ignored but historically important region would get $\alpha * H_{\text{hist}}$ worth of heat once and then stay there forever, because nothing feeds `H_final` back into `H_sim`. So `H_sim` is partly a function of the previous tick's population, since people attract trade, labor, and markets:

$$H_{\text{sim}}(t+1) = \text{normalize}(\text{drivers}(t+1) + \beta * (\text{Pop}(t) / \max \text{Pop}(t)) ^ (1/k))$$

Here **drivers** is the gameplay term (food, water, trade, infrastructure, war) and β is how strongly existing population draws more. Taking the $1/k$ root converts population back to heat scale, so the feedback carries population's momentum without adding concentration of its own. k stays the only concentration knob. At the 300 BC start the feedback term equals **H_hist**, so the historical starting state is a fixed point: nothing drifts until play changes the drivers. The small historical boost from α then compounds into a real drift toward history, and heavy, sustained player investment in **drivers** can still beat it. That's the "realist alternate history" behavior. Explicit relaxation of **H_sim** toward **H_hist** was the alternative. It was rejected because it drags on the simulation directly instead of acting through population.

$\beta = 5.0$, together with population inertia $\mu = 0.065$ (stage 2). Both are calibrated to real founded cities on the real HYDE grid. (decided; μ refitted from 0.0625 once regional totals were built, see Fit) Feedback alone can't make history slow to bend. The update is a contraction whose per-tick factor is at most $1 - \alpha$, so any change in drivers settles within a few years whatever β is. A simulation showed a barren node turning into a major city within 0-1 years. The timescales come from population being a stock (see stage 2). The pair (β, μ) is then fitted to how long real non-capital cities took to climb the rankings. Capitals get their own calibrated bonus on top (see Capital growth bonuses, below).

Historical target. Years from a city's founding until it entered the world's top 50 and top 10 cities by population. The figures are rough estimates from Chandler (*Four Thousand Years of Urban Growth*) and Modelski. The sample is cities that did make the list; most foundings never do. "Capital" means the city was a country's capital from founding or within its first few decades.

CITY	FOUNDED	CAPITAL?	→ TOP 50	→ TOP 10
Alexandria	331 BC	yes	~20	~50
Seleucia-on-Tigris	305 BC	yes	~25	~55
Antioch	300 BC	yes	~50	~100
Pataliputra	490 BC	yes	—	~190
Constantinople (refounded)	330 AD	yes	~10	~60
Baghdad	762	yes	~10	~30
Samarra	836	yes	~8	~15
Edo	1603	yes	~20	~50
St. Petersburg	1703	yes	~50	~160
Calcutta	1690	no	~110	—
New York	1624	no	~190	~226
Philadelphia	1682	no	~150	~210

CITY	FOUNDED	CAPITAL?	→ TOP 50	→ TOP 10
Chicago	1833	no	~37	~57
Melbourne	1835	no	~50	—
Los Angeles	1781	no	~140	~170
Shenzhen	1980	no	~20	—
Average, capitals			24.1 → 24	78.9 → 78
Average, non-capitals			99.6 → 100	165.8 → 166
<i>Average, all</i>			59.3 → 60	105.6 → 106

Targets are the averages rounded to the nearest even number. The all-cities average (60/106) was the first calibration. It blended two different populations: capitals founded by decree and organically grown commercial cities. So the base engine is now fitted to **non-capitals (100/166)**, and capitals reach **24/78** through their own bonus.

Fit. `python3 tools/calibrate_population.py --hyde` simulates every land node of the real baked 300 BC keyframe (284,626 nodes), `k = 2`, yearly ticks, history held at 300 BC. It measures the median populated node given drivers matching the world's best site and held there. Ranks are within the map's extent, not the whole planet, so the map region's rank-size curve is what matters. A coarse-then-fine grid search gives `β = 5.0`, `μ = 0.0625` → **top 50 at year 102, top 10 at year 168**, the closest fit found on one map-wide pool.

Refit on the regional engine (built, 25 Sep 2026). Once each geographic region kept its own total (see Growth drivers), a new city drew only on its own region, and the same constants gave 102/172 years. A grid search on the regional engine itself, on the real grid with the 38 regions, kept `β = 5.0` and moved `μ` to `0.065`: **top 50 at year 98, top 10 at year 165**, against the historical 100/166, closer than before.

`src/Tests/PopulationHydeTest.cs` holds this fit and checks the engine reproduces it.

`tools/calibrate_population.py` still models the single pool and the toy world.

HYDE's top is flat: at 300 BC the #10 node holds ~20,000 and the #50 ~15,000, so a city that passes #50 is close to #10. Getting the historical 66-year gap between the two needs a city whose growth slows sharply as it nears the top. A large `β` does that: existing population, not drivers, dominates heat, so a newcomer's pull weakens against the established giants. The fast `μ` then gets it to the top 50 on time.

The constants were first fitted on a synthetic toy world (5,000 nodes on an ancient rank-size curve, population $\propto \text{rank}^{-0.6}$): `β = 0.25`, `μ = 0.0115`, 96/166 there, but 143/165 on the real grid. The toy world is kept only as a fast test fixture.

Behavior at these values, measured on the real grid (the old toy-fit values in brackets):

SCENARIO	RESULT
Barren median node, drivers maxed and held indefinitely (player only; see Historical ceiling)	Top 50 at 102 years , top 10 at 168 years (98 and 165 on today's regional engine). Levels off at ~21% of the largest city's population, the map's #8 [62%, #2]
Same city, investment abandoned after it matures	Half its growth (measured on a log scale) is gone in ~73 years [104]
6th-largest historical city loses all its drivers for 30 years	Loses ~37% of its population [49%] and is back to 90% of its former size ~72 years after the war ends [191]. Established cities are sturdier: their own population holds their heat up

History's pull at these values comes from α , from the large β (established places keep their weight), and, for everyone except the player, from the hard Historical ceiling below.

2. HEATMAP-TO-POPULATION TRANSLATION

```

Target_node   = C * H_final ^ k
Pop_node(t+1) = Pop_node(t) * (Target_node / Pop_node(t)) ^ μ      μ = 0.065 /
year

```

Population is a **stock**, not recomputed from scratch each tick. People can only be born, die, and migrate so fast. Each year a node closes ~6.5% of the gap to its target, **measured on a log scale**: growth and decline are percentage rates, like real demography. A town 100x below its target grows ~33% a year, a gold-rush boom that fades fast as it closes in. A city 10% below its target grows ~0.7% a year. The log-scale form is required, not stylistic. A linear step ($\text{Pop} += \mu(\text{Target} - \text{Pop})$) grows fastest at the very start, which makes a founded city's climb from top 50 to top 10 take at least ~2.6x as long as reaching the top 50, versus ~1.8x historically. Compounding growth matches the real curve. It's what makes bending history a long-term strategy rather than something one build order does. Each node's own **Pop** is the value everything else reads (Might, taxes, naming, the feedback term above). **(DECIDED)**

Populations are whole people. Starting and seed populations come from HYDE; see Starting and seed populations, below. **(DECIDED)**

This is a power law, not exponential decay, but it has the intended effect: a higher **k** crushes middling heat far harder than peak heat, so population funnels into the hottest nodes.

- **C (carrying capacity, era multiplier)** scales with technology unlocks: about **150,000** for ancient megacities at the 300 BC start, and **tens of millions** by the industrial and space eras. It's implemented in world-total form (see below), so those per-city numbers are calibration checks on the hottest node, not a literal constant.
- **k (urbanization exponent)** rises with era. **k = 2** in ancient and agrarian eras spreads population out. **k = 4-5** in modern and interstellar eras simulates metropolitan concentration.

(DECIDED)

Two consequences worth knowing before tuning:

- **C is a world total, not a per-node ceiling. (decided)** Because $H \leq 1$, $H^4 \leq H^2$ everywhere. With **C** as a literal per-node ceiling, an era transition from $k = 2$ to $k = 4$ would make every node except the peak lose population overnight: a node at $H = 0.5$ would drop from $0.25 \cdot C$ to $0.0625 \cdot C$. So the implemented form is

$$\text{Target}_i = P_{\text{world}} * H_i^k / \sum_j H_j^k$$

P_world is the era's world population: HYDE's historical total by default. Growth mechanics (see Growth drivers) can push it off history, but only the player's realm can push it *above* history. Bots' growth mechanics can only lower it (see Historical ceiling). The total is preserved by construction, **k** still funnels people into the hottest nodes, and the output can be checked directly against HYDE's totals. The ~150,000 ancient-megacity figure becomes a sanity check: at 300 BC with $k = 2$, the hottest node should come out around there. - **k must change gradually**. A step change in **k** at an era boundary redistributes the whole world in one tick. Interpolate **k** (and **C**) over the transition, the same way **H_hist** is interpolated between keyframes.

HISTORICAL CEILING: ONLY THE PLAYER EXCEEDS HISTORY

Bots and unowned land can never hold more people in a place than real history had there at that date.

Only the player can. (decided) Every node has a ceiling that tracks history as it happened, rises *and* falls:

```
Ceiling_i(year) = HYDE_pop_i(year)      (interpolated between snapshots, like
H_hist)
```

When real Babylon declines after Seleucia is founded, its ceiling declines with it, and a bot-held Babylon declines too. A bot can't keep it at its peak. Together with the α pull, which draws *under*-populated places up toward history, bot-held and unclaimed land hugs the historical record: never above it, and pulled back up when it falls below. (An earlier draft used the running historical maximum as the ceiling. That was wrong. It let a bot freeze Babylon at its peak forever, which contradicts the whole point of the historical pull.) Rules:

- **Applies to:** every node not owned by the player's realm. That covers bot realms, unorganized land held by bots, and unclaimed land.
- **Enforced on the population itself:** $\text{Pop}_i \leftarrow \min(\text{Pop}_i, \text{Allowance}_i)$ after each tick's update, where $\text{Allowance}_i = \text{Ceiling}_i$ for any place the player never lifted above history. Capping only the *target* isn't enough. Population closes about 1.2% of its log gap to target each year, so when history falls it would lag far behind. Babylon's roughly 20x fall over three centuries would leave a bot-held Babylon about 2.4x above history the whole way down. Clamping the population makes bot-held places follow history's own decline at history's own pace. Targets are clamped too, so growth toward an unreachable target doesn't spill into the redistribution below.
- **Player legacy fades instead of snapping:** when a node the player held above its ceiling passes out of player ownership (conquest, or abandoning the land), **Allowance_i** starts at the node's population

at that moment and relaxes toward **Ceiling_i** on a log-scale curve of its own, $\mu_{\text{legacy}} = 0.0115$: about 60 years to halve the excess on a log scale. It's kept apart from μ , which the growth fit set to 0.065; at that rate a lost city would be back to history within a generation. The city shrinks back to history gradually rather than overnight. Once **Allowance_i** reaches the ceiling, the node is an ordinary capped node again.

- **Capped excess stays with bots:** population trimmed off capped targets is redistributed to other *non-player* nodes still below their ceilings, in proportion to their targets. This keeps the world total on HYDE's historical figure. The player's own targets are never touched by it, so the player neither gains nor loses from bots hitting their caps. If every non-player node is at its ceiling, the excess is simply never born.
- **World total:** see **P_{world}** above. Only the player's realm can lift it above history.
- **Freedom below the ceiling:** bots still diverge from history *downward* and in how population shifts between places under their ceilings (wars, lost trade, moved capitals). They just can't create a metropolis where history never had one. Alternate-history *growth* is the player's alone.

A consequence worth knowing: a bot can't found a large new city in a place that was historically empty, since the ceiling there is near zero. Bots found cities where history did. The player is the only realm that can put a great city somewhere new.

CAPITAL GROWTH BONUSES

Applies to **country capitals** (a realm's seat) and **province capitals** (each organized province's administrative seat), for both the player and bots. The ancient geographic regions (see Geographic regions) are purely geographic and have no capitals. For bots, the bonus only speeds growth *up to* the historical ceiling; it never lifts them past it. **(DECIDED)**

Two parts, both historically grounded, fitted to the targets above:

	ONE-TIME RESETTLEMENT WHEN DESIGNATED	ONGOING DRIVER BONUS	→ TOP 50	→ TOP 10	HISTORICAL TARGET
Country capital	8% of the largest city's population	+0.05	32y	76y	24 / 78
Province capital	4% of the largest city's population	none	60y	126y	62 / 122
Ordinary city	—	—	98y	165y	100 / 166

Fitted on the real HYDE grid with the base β , μ above (see Fit), measured on the regional engine. The country capital's top-10 time is 2 years early; its top 50 lands 8 years late, the closest the pair gets.

- **Resettlement** reproduces the founding decree. Seleucia was populated from Babylon, Samarra from Baghdad, and Antioch started with ~5,300 Athenian settlers from Antigonía plus their families. At 300 BC the largest node holds ~108,000, so 8% is ~8,600 people for a country capital and 4% is ~4,300

for a province capital, the size of a Roman veteran colony. The settlers are **moved, not created**: they're drawn from the realm's other nodes in proportion to their population. It happens only the first time a given city becomes that realm's capital of that kind, so toggling capitals can't farm settlers. For bots it's clamped to the capital's remaining headroom under the ceiling.

- **Driver bonus** stands for the court, bureaucracy, garrison, and tribute spent at the seat. It's added to the node's **drivers** in the **H_{sim}** feedback formula for as long as the city stays capital. When it stops being capital, the bonus goes away and the city declines along the normal **μ** curve, which is the Samarra and Pataliputra arc. On the real grid the fit wants it small (+0.05 for a country capital, none for a province capital): resettlement does nearly all the work.
- A faster growth rate (a **μ** multiplier) was tested as the bonus instead. It can't reproduce capitals' fast start: its best fit was 45 years to the top 50 against a target of 24. Resettlement is what produces the historical jump.
- **Province-capital targets are interpolated**, midway between country capitals and ordinary cities. A founding-to-rank dataset for provincial seats doesn't exist the way it does for world top 50 and top 10. Rough anchors support it: Basra and Kufa, garrison capitals founded 636/638, reached the top tier within ~30 years; Roman Carthage, a provincial capital refounded in 44 BC, took about 50 years to the top 50 and ~130 to the top 10; Fustat and Lugdunum took longer.

(RETUNE WHEN BETTER PROVINCIAL DATA TURNS UP)

- **Capitals at the 300 BC start** already carry their bonus, and no resettlement is triggered for them. Starting drivers are backed out from HYDE so the historical start is still a fixed point with every existing capital's bonus included. Only capitals moved or founded after the start change anything.

STARTING AND SEED POPULATIONS

Everything starts from the historical population heatmaps (HYDE). (decided)

- **Game start**: every node's population is its HYDE population for 300 BC. The whole world begins as historical record, not procedural.
- **Seeding an empty node** (a founding on empty land, or the resettlement of a ruin) uses, in order: the node's HYDE population for the current year; else, for the player only, the median rural node density of its ancient geographic region. For bots and unclaimed land the seed is clamped to the ceiling. Where history had nobody that year, nobody but the player can settle. A new settlement starts at what history says that land held. Capital resettlement, above, adds to that seed.
- **Ruins need an event**. Log-scale growth never reaches zero on its own while a node has any target. So a settlement only becomes a ruin through an explicit depopulating event: razing, plague, or forced evacuation. A ruin stays empty until a realm deliberately re-founds it (reseeded immediately), or until a generation, **25 years**, passes and it's naturally resettled at its seed size. Resettlement doesn't restore the name. The ruin revives under its old name only when its cluster re-enters the naming percentile. Carthage was razed in 146 BC and refounded by decree in 44 BC. (**25 years is a starting value**)

GROWTH DRIVERS: WHAT MAKES PEOPLE, AND WHAT MOVES THEM

(decided: approved as drafted, 25 Sep 2026; built, with every factor at its historical baseline: see As built, below)

Why this exists. The engine is calibrated for how fast population *responds* to **drivers**. Nothing yet defines what **drivers** are. Today every node's drivers are its historical heat and the gameplay slot (**driver_mods**) is zero, so the world simply follows HYDE. The one calibrated case is a single site given drivers of 1.0, the best site on the map. A stress test on the real 300 BC grid shows what an undefined driver formula would do. A contiguous, roughly Greece-sized realm (4,082 nodes, 2.3 million people) with every node at drivers 1.0 holds **21.0 million of the map's 39.5 million after 200 years**, and the largest city outside it falls from ~108,000 to ~24,000. At drivers 3.0 the rest of the map keeps 0.9 million. A single boosted city is harmless; broad, stacked drivers across a realm are not, and a player investing everywhere produces exactly that. The old toy-world constants break the same way, so this is structural, not a calibration problem. Three causes:

- **One map-wide pool.** **P_world** is fixed and heat is normalized by the map-wide maximum, so anything that raises one realm's heat lowers everyone else's. Attraction becomes a vacuum.
- **Nothing bounds drivers.** If factor effects simply add up, they stack without limit.
- **"Growth" and "movement" are the same number.** The engine has one heat term that decides both how many people there are and where they live, so it can't tell a baby boom from an exodus. The log-scale step also doesn't conserve people under strong drivers: moving every node a fixed share of its log gap doesn't preserve the sum, and the same stress test lost 2-4 million people that way, with nobody dying.

Principle: two channels, each bounded. (decided)

1. **Natural increase** (births minus deaths) decides *how many* people a region has. It's computed per ancient geographic region (see Geographic regions), from many factors, not just food.
2. **Attraction and migration** decides *where* they live. It conserves people exactly: moving someone never creates or destroys them.

P_world becomes the sum of the regional totals, replacing the single historical figure plus **player_world_delta** (removed).

CHANNEL I: NATURAL INCREASE, PER REGION

$$P_r(t+1) = P_r(t) * (1 + g_r)$$

$$g_r = g_{hist_r}(year) + \sum_f w_f * s_{f,r} \quad (\text{then bounded, below})$$

g_hist_r is the region's own historical growth rate, read from the HYDE keyframes. Each factor score **s_f,r** runs from -1 to +1 and is measured **against that region's historical baseline**: 0 means "as history had it". So a region nobody touches grows exactly as history did, and every factor is a deviation from the record, not an absolute rate the designers have to guess.

Real pre-modern growth was never one variable. Food mattered, but so did disease, violence, cities, burdens and luck. Factors:

FACTOR	WHAT FEEDS IT	DIRECTION AND SHAPE	HISTORICAL GROUNDING
Food security	Harvest (arable land x agriculture tech x irrigation x climate) against consumption, plus storage (granaries) and food imports along trade routes	Asymmetric: a deficit hurts far more than a surplus helps	Rome fed itself on Egyptian and African grain; famine years spiked mortality, good years barely moved fertility
Disease burden	Density, share of people in cities, sanitation (clean water, aqueducts, sewers), marshland and malaria zones, and trade connectivity, which spreads epidemics	Mostly negative; sanitation and medicine reduce it	Antonine Plague (165), Plague of Justinian (541), Black Death (1347) all travelled trade routes; the Pontine marshes and lower Mesopotamia were malarial
Urban penalty	Share of the region's people living in large nodes	Negative until sanitation and medicine tech offset it	Pre-modern cities had more deaths than births (the "urban graveyard"); they grew only by migration from the countryside
Security	War fought on the region's land, raids, banditry, occupation; peace and strong garrisons	War is strongly negative (burned harvests, flight, famine); long peace is mildly positive	The Pax Romana's growth; the third-century crisis and the Gothic wars in Italy
Land and living standards	Land per person against carrying capacity, wages, access to new land	Positive when land is plentiful (earlier marriage, more surviving children); falls to zero as the region nears capacity	Frontier and colonial populations grew fastest; crowded old regions stalled (the Malthusian trap)
Burden	Taxes and tribute, forced labor, slavery, serfdom, concentration of land into large estates	Negative, mild to moderate; heavy burden also pushes people to leave (channel 2)	Peasant flight from over-taxed land in the late Roman Empire
Health and public institutions	Medicine and public-health tech, famine relief, poor relief	Positive, era-gated; small in antiquity, large in modern eras	The mortality decline of the 1700s-1800s
Climate and harvest luck	A random yearly shock per region, not player-controlled; rare large events	Either sign; occasional severe negatives	The volcanic winter of 536; the Little Ice Age

Epidemics, famines and massacres are **events**, not factor scores. They remove a share of a region's people directly (up to ~35% for a Black Death-scale plague, spread over a few years) and are outside the rate bounds below. Their odds are what factors like disease burden and trade connectivity raise. Culture and family structure (for example later marriage) could become a small civ trait. **(OPEN)**

Carrying capacity. Every positive part of g_r is scaled by $(1 - P_r / Cap_r)$, where Cap_r is the region's arable land times the yield its agriculture tech allows (from the Resources density field). A region can approach its capacity but never grow past it; only better agriculture, irrigation or imported food raise it. That's the Malthusian ceiling that held every pre-modern region.

Bounds, anchored to HYDE. (decided; values retuned in balance tests) On this map, HYDE's century-averaged regional growth (5-degree regions) before 1700 runs from **-0.7% to +0.6% a year**, and the whole map from -0.2% to +0.23%. Only from 1700 do regions exceed +0.8%. So:

- **Factor deviation** $\sum w_f s_{f,r}$ is clamped to **-2.0% to +0.5% a year** in the ancient and medieval eras. At +0.5% a year, a region can grow ~2.7x more than history over 200 years, a very strong alternate history but not a runaway. The upper bound rises by era with health tech.
- **Bots:** natural increase can only fall below history, never exceed it (only the negative part applies), which keeps the Historical ceiling rule. The player's regions get both signs.
- **Weights w_f :** a first proposal is to split the positive range across food 30%, land and living standards 25%, security 20%, health 15%, burden 10%. Disease, urban penalty and climate act on the negative side. These are starting values, to be set by the balance tests below. **(OPEN)**

CHANNEL 2: ATTRACTION AND MIGRATION (WHAT **drivers** BECOMES)

drivers is redefined as **attractiveness**: how much a place pulls people from elsewhere. It's bounded and saturating:

$$\begin{aligned}
 A_i &= \sum_f v_f * a_{f,i} && \text{(pull minus push)} \\
 \text{drivers}_i &= h_{\text{hist}_i} + (cap_{\text{era}} - h_{\text{hist}_i}) * \text{sat}(A_i) && \text{if } A_i \geq 0 \\
 \text{drivers}_i &= h_{\text{hist}_i} * (1 - \text{sat}(-A_i)) && \text{if } A_i < 0 \\
 \text{sat}(x) &= 1 - e^{-x}
 \end{aligned}$$

With $cap_{\text{era}} = 1.0$ in the ancient era, no place can become more attractive than the best site on the map, which is exactly the case the calibration measured. Stacking has diminishing returns: past a point, another road or market adds almost nothing. Pull and push factors:

- **Pull:** economic opportunity (trade routes, markets, ports, crafts, mines and other resource jobs), the seat of government (the existing capital bonus folds in here), security (walls, garrisons), public works (roads, water supply, harbors), open land for colonists (Greek colonies, Roman veteran colonies), and religious draw (sanctuaries, pilgrimage), a small one.
- **Push:** war and occupation, famine, epidemic, persecution, heavy taxes and tribute, loss of land to large estates.

Migration conserves people and is limited. (decided)

- **Within a region**, attraction redistributes the region's people the way the engine does now, heat to target to the log-scale step, but the region's total is renormalized after every step so it stays exactly P_r . That also closes the leak described above.

- **Between regions**, a net flow runs from regions whose average pull is below the map's toward those above it. It's capped at **0.2% of the source region's people a year**, and only between regions that share a border or a sea lane. Large historical streams (Greek colonization, the Germanic migrations) fit inside that over decades.
- **Forced movement** (deportation, resettlement of a conquered people, the Assyrian and Babylonian practice) is an explicit player action with its own limit, like the capital resettlement.

Prototype on the same stress test, with 5-degree blocks standing in for the regions until the region mask is built:

GREECE-SIZED REALM, MAXED DRIVERS, 200 YEARS	REALM (2.3 M AT START)	REST OF THE MAP
Today: one map-wide pool	21.0 M	8.2 M
Regional totals	2.9 M	35.1 M
Regional totals + capped migration	3.0 M	35.0 M

The realm still gains, about 1.3x, but by drawing on its own region and a limited stream from its neighbours, not by emptying the map. Anything beyond that has to come from channel 1, which is bounded separately.

Calibration impact. The founded-city timings were measured with one map-wide pool. With regional totals, a new city draws only on its own region, so β and μ were refitted on the regional engine: μ is now 0.065 (see Fit). **(done)**

BALANCE TESTS: THE CONTRACT

Each test runs on the real HYDE grid and must pass whenever a constant, weight or factor formula changes. They go into `tools/calibrate_population.py` and the headless test suite alongside the founded-city calibration. **(DECIDED; BOUNDS RETUNED IN BALANCE TESTS)**

TEST	SETUP	MUST HOLD
Do nothing	Player realm, every factor at its historical baseline, 300 years	Tracks HYDE within $\pm 5\%$
Maxed attraction	Greece-sized realm, every pull factor maxed, 200 years	Realm $\leq 1.5x$ its historical population from migration alone; no other region loses more than 10% to it
Maxed growth	Every natural-increase factor maxed, 200 years	Realm $\leq 2.7x$ history (the +0.5% a year bound) and never above its carrying capacity
Stacking	Double every factor input after saturation	Drivers change by $\leq 10\%$
Catastrophe	Worst war, famine and epidemic together for 10 years	Region loses $\leq 50\%$, and is back to 90% of its former size within 150 years of peace
Conservation	Migration only, natural increase off	Map total unchanged to within one person

TEST	SETUP	MUST HOLD
Founded cities	The existing calibration	Timings stay near the historical targets

These bounds are what keeps a future economy, tech tree or resource system from breaking population: any system can raise a factor score, but none can push past what these tests allow.

AS BUILT (25 SEP 2026)

The groundwork is in: regional totals, conserving migration and the balance-test harness, with every factor at its historical baseline. No gameplay system writes a factor score yet; each will be wired in as it's built, and must pass the balance tests first. Code: `src/World/PopulationEngine.Growth.cs` (both channels) and `src/World/PopulationEngine.Regions.cs` (the region layer). A few details the draft left open were settled while building it:

- **History as the baseline, exactly.** A region's natural increase is HYDE's own year-on-year change for that region, times $(1 + \text{deviation} + \text{recovery})$. With nothing changed, every region follows HYDE: over 300 years the worst region is within 0.12% of it.
- **Factor weights** are the proposal's: the +0.5% a year is split food 30%, land and living standards 25%, security 20%, health 15%, burden 10%. The -2% a year is split food 25%, security 25%, disease 20%, urban penalty 10%, burden 10%, land 5%, climate 5%. **(starting values, open)**
- **Mixed regions.** A region can hold the player and bots. Only the player's share of its people gets the positive part of the deviation. Bot-held nodes are still capped at history; growth that doesn't fit under their caps goes to the player's nodes in the region, but only as far as the region is above history.
- **Carrying capacity** is provisional: 3x the region's historical population, until the Resources density field gives each region its arable land and yield. Positive growth fades to zero as a region approaches it. **(OPEN: REPLACE WITH ARABLE LAND X YIELD)**
- **Recovery (new).** A region knocked below history regrows toward it at 1.5% of the log gap a year, never faster than +1% a year: empty land and plentiful food after a catastrophe, the land-and-living-standards factor's baseline. Without it a region halved by a plague would stay halved forever, since its historical rate only preserves the ratio.
- **Migration between regions** runs from a region toward neighbours with stronger pull (a shared border or a sea lane within 300 km). A region's pull is its people's average attraction. People keep moving until crowding balances the pull: at full pull a region settles at about 1.4x the people-to-history ratio of its neighbours ($\text{MigrationPull} = 0.35$). A region without the player can't take in more than its historical ceiling has room for.
- **Attraction and drivers.** Each node's attraction (`DriverMods`, pull minus push) saturates as drafted: $\text{drivers} = \text{base} + (1 - \text{base}) * \text{sat}(A)$. The capital bonus is added after that, so the calibrated capital timings still hold.
- **Whole people only.** HYDE gives many desert and steppe nodes less than one person. Those start empty, so regional history counts whole people only; otherwise sparse regions (Libya Interior,

Sarmatia) looked permanently under-populated, drew people in, and couldn't hold them. An empty node gets a share of its region's people only once it's due to resettle.

- **Events** (epidemic, famine, massacre) remove up to 35% of a region's people per call (**ApplyShock**), outside the rate bounds. Nothing triggers them yet.

Balance tests, first results (`src/Tests/PopulationBalanceTest.cs`, real grid, all pass; founded cities are `PopulationHydeTest`):

TEST	RESULT	LIMIT
Do nothing (300 years)	Worst region 0.12% off HYDE, map total 0.008%	±5%
Maxed attraction (Hellas, 200 years, migration only)	Hellas 1.37x; hardest-hit neighbour (Phrygia) lost 3.0%	≤ 1.5x; ≤ 10%
Maxed growth (200 years)	Hellas 1.73x; a bot region with the same factors 1.0000x	≤ 2.7x and capacity; bots never above history
Stacking	Doubling a saturated attraction changes drivers by ≤ 5.0%	≤ 10%
Catastrophe (Italia: every factor at worst plus a 34% plague, 10 years)	Lost 39.4%; back to 90% 62 years after peace	≤ 50%; ≤ 150 years
Conservation (migration only, 100 years)	39.4 million held to within 6 people	one person per million (32-bit storage)
Founded cities	Ordinary 98/165, country capital 32/76, province capital 60/126	near 100/166, 24/78, 62/122

The conservation limit is one person per million rather than one person: each node stores its people as a 32-bit number, like HYDE and the saves, and adding 285,000 of them can't be exact to the person. The engine's own bookkeeping moves people exactly.

Build order. Channels and tests come before any system feeds them: regional totals, conserving migration, and the balance-test harness first, with every factor at its historical baseline (which must reproduce today's behavior). Then each gameplay system (resources, economy, tech, military) is wired to one or more factor scores and must pass the whole table before it merges.

3. SETTLEMENT NAMING AND SPAWNING (PERCENTILE-BASED)

- **Spawn:** a cluster becomes a named settlement when its population enters the **top 2-5%** of all active population clusters (population > 0; empty clusters are excluded so a large empty map doesn't shift the bar). This keeps the label count proportional to map size in every era.

(DECIDED — 3% TO START; RETUNE WITHIN 2-5% IN PLAYTESTING)

- **Hysteresis:** once named, a settlement keeps its name, even when conquered or when its land becomes unclaimed, until it drops below the **top 20%**. The wide gap between entry (2-5%) and exit (20%) stops labels flickering on and off when a city hovers near the line. (DECIDED)

- **Ruins:** a named settlement whose population reaches exactly zero loses its active status but becomes a **ruin**: name and position are kept as a map marker, not deleted. It can be re-founded. Only depopulating events cause this; see Starting and seed populations. (DECIDED)
- **Scale shift:** at the planetary-to-interstellar transition the rule doesn't change, only what a "cluster" is. The top 2% of hexes becomes the top 2% of hemispheres or core worlds. (DECIDED)

Useful property: **the ordering of target populations depends only on the ordering of H_{final} , not on C or k** . A percentile rank is unchanged by any increasing transform, and $C * H^k$ is increasing in H . So retuning C and k changes displayed populations but never the settlement ranking the world is heading toward. Only the heatmap changes that. (Actual populations lag their targets by the inertia above, so the names on the map settle into that ranking over decades, not instantly.) Calibration and map readability can be tuned independently.

Cluster vs. node vs. cell. (decided) The percentile has to rank *clusters*, not grid cells. The ownership grid is 8192x5476 at ~0.59 km² per cell, so there are millions of land cells, and the top 2% of *cells* would be hundreds of thousands of labels. A single city also covers many adjacent cells. So:

- **Node:** the unit H and Pop are computed on. Coarser than a cell, roughly HYDE's native resolution (5 arcminutes, ~9 km), since there's no finer truth to compute against. Per-cell density values are sampled from their node.
- **Cluster:** a local maximum of H_{final} plus the contiguous nodes that drain to it (a watershed on the heatmap). Its population is the sum over those nodes. Nearby peaks closer than a minimum separation merge, so twin peaks don't produce two labels for one city.

Global ranking with a regional floor. (decided) A purely global percentile would put almost every 300 BC label in the Aegean, the Nile, Syria, and Mesopotamia. That's historically honest, but it leaves regions like Libya Interior or Scythia with no labels at all. So the ranking is global, plus a regional floor: any region with no named settlement gets its single largest cluster named. Hysteresis and ruin rules apply to floor-named settlements too. Regions are a fixed geographic partition, not political borders, so conquest never changes which floor applies.

GEOGRAPHIC REGIONS — THE ANCIENT GEOGRAPHERS' MAP (BUILT)

The partition uses the regions the Greco-Roman geographers themselves drew (Hecataeus, Herodotus, Eratosthenes, Strabo, Ptolemy), grouped under their three continents: **Europa**, **Libya** (their name for Africa), and **Asia**. The map's extent (longitude -10 to 55, latitude 10 to 48) is almost exactly the world those geographers described, so their divisions cover it without inventing names. Continent borders follow the ancient convention: the Tanais (Don) between Europa and Asia, the Nile/Isthmus line between Libya and Asia. (DECIDED)

CONTINENT	REGIONS
Europa	Hispania, Gallia, Raetia, Italia (with Sardinia and Corsica), Sicilia, Pannonia, Illyricum, Dacia, Thracia, Macedonia, Hellas (with Crete and the Aegean islands), Scythia

CONTINENT	REGIONS
Libya	Mauretania, Numidia, Africa (the Carthaginian heartland, the name the continent itself later took), Syrtica, Cyrenaica, Marmarica, Aegyptus, Libya Interior (the Sahara), Aethiopia (everything south of Egypt, Kush included)
Asia	Lydia, Phrygia, Cilicia (with Cyprus), Cappadocia (with Pontus), Colchis (the Caucasus), Armenia, Syria (with Phoenicia and Judaea), Mesopotamia (with Assyria), Babylonia, Arabia Petraea, Arabia Deserta, Arabia Felix, Media, Susiana, Persis, Hyrcania, Sarmatia

That makes 38 regions. Smaller neighbours are folded into the larger unit named above rather than given their own entry. As with Europa, several of these names are the ancient roots of modern ones (Africa, Arabia, Armenia, Syria, Libya).

As built (25 Sep 2026). `tools/build_region_mask.py` bakes a region id for every population node into `data/regions/` (`region_nodes.u8.gz` and `regions.json`, which lists each region's continent, what's folded into it, where its name goes, and its neighbours). Each region is a hand-drawn outline in longitude/latitude whose borders follow the ancient descriptions snapped to physical features: the Pyrenees, the Alpine crest, the Danube and Sava, the Nestos, the Tanais (Don), the Caucasus, the Taurus, the Euphrates, the Zagros, and the Nile/Isthmus line. The coastline comes from the land data, not the outlines. Islands go whole to their region (Lesbos, Rhodes and the other east Aegean islands to Hellas although they lie off Asia's coast; Cyprus to Cilicia; Sardinia and Corsica to Italia). What each region takes in, where the table above doesn't say:

- Raetia takes Noricum and the strip north of the upper Rhine (the map's only bit of Germania); Gallia takes the Helvetii.
- Illyricum takes Dalmatia, Dardania and Upper Moesia; Thracia takes Lower Moesia and the Dobruja; Dacia runs to the Tyras (Dniester) and takes the Tisza plain; Hellas takes Epirus.
- Lydia takes Ionia, Aeolis, Mysia, the Troad, Caria and Lycia; Phrygia takes Bithynia, Galatia, Pisidia and Lycaonia; Cilicia takes Pamphylia; Cappadocia takes Paphlagonia; Colchis is the whole Caucasus (Colchis, Iberia, Albania).
- Hyrcania takes Parthia and the Dahae east of the Caspian; Sarmatia is everything beyond the Tanais north of the Caucasus and the Caspian; Persis takes Carmania.
- Aethiopia is everything south of Egypt, and the Sahel south of the 16th parallel; Libya Interior is the Sahara between.

Two regions are neighbours if their land touches or a sea lane joins them (coasts within 300 km across water); migration only runs between neighbours. Borders are exact to the population grid (~9 km), which is the finest data the regions are used on. **(boundaries can be redrawn in the tool and re-baked; regions beyond the current extent, if the map grows, get names the same way, from the geographers who described those lands)**

Display: only on the "Regions" map overlay. (decided, built) Region names never appear on the default map view. They show only when the player switches on the **Regions** overlay (the Regions button, or the R key), which tints each region in its continent's colours (Europa blues, Libya ochres, Asia reds), draws a faint boundary between regions and their names, and hides realm borders and names while it's on. The province panel also names the region a province lies in. The regional-floor rule runs all the time regardless of which overlay is showing.

RESOURCES AND THE LAND (PROPOSED, 25 SEP 2026)

The designer's resource plan, with suggestions folded in. Nothing here is built yet; the questions marked **(OPEN)** are in the Ledger.

THREE TIERS, NOT ONE LIST

1. **The land** (what a place *is*): soil, water, climate, terrain, living habitat, geology. Stored per population node (~9 km, 284,626 land nodes), the same grid as population and regions. That's fine enough for fields, forests and ore bodies and small enough to simulate: 40 numbers per node is ~45 MB. The 45-million-cell ownership grid is too fine to simulate and has no finer data behind it.
2. **Raw resources** (what a place *can yield*): crops, orchards, herds, fish, timber, stone, clay, ores, salt. Each is a *potential* per node, computed from the land layer by a suitability profile, times how much of it is actually worked (people, tools, tech).
3. **Goods** (what people *make*): flour, bread, beer, olive oil, wine, yarn, cloth, charcoal, bricks, pottery, bronze, iron, tools, weapons, ships, and the prestige goods. These aren't on the map at all: they're **recipes** (inputs, labour, fuel, a workshop or skill) run where the inputs and the people are. That keeps the map data small and makes the full goods list cheap to add to.

Provinces and realms sum all three, as they already do for population (see Layers: stats are always summed, never stored on the province).

THE LAND LAYER

The designer's six parts, each a handful of numbers per node:

PART	NUMBERS PER NODE	DRIVES
Soil	depth, texture, drainage, organic matter, N/P/K reserve, salinity, acidity, erosion, compaction	which crops thrive, yields, how fast farming wears a field out, whether drainage or manure helps
Water	seasonal rainfall, river flow, floodplain moisture, springs, groundwater depth, water quality, irrigation supply	crop survival, herd size, gardens and orchards, settlement capacity, mills, river transport
Climate	growing season, heat, frost, drought frequency, prevailing winds, storm exposure	crop suitability, harvest reliability, animal survival, sailing seasons
Terrain	slope, elevation, aspect, flood risk, passability	usable farmland, terracing cost, grazing, roads, mines, transport
Living habitat	shares of grassland, woodland (with species mix and age), scrub, wetland, river habitat, coastal nursery grounds	forage, timber, reeds, honey, medicinal plants, fish and shellfish

PART	NUMBERS PER NODE	DRIVES
Geology	ore bodies (grade, size, depth), workable stone, clay beds by use, salt, sand, minerals	mining, building, metalworking, pottery, glass, pigments

Suggestion: bake it from real data, the way HYDE gives population, so the map is history, not invention. Candidates: soils from HWSD/SoilGrids; climate from WorldClim or CHELSA (with CHELSA's palaeoclimate for 300 BC); elevation and slope from the relief data the terrain already uses; rivers from HydroSHEDS; 300 BC forest cover from the KK10 reconstruction (which models ancient deforestation); ores from the USGS mineral database plus the archaeological atlases of ancient mines and Roman quarries. HYDE itself has cropland and pasture grids for 300 BC: a direct check that the farming model puts fields where history had them. Each becomes a `tools/build_*.py` step, like the region mask.

EACH CROP AND HERD ITS OWN PROFILE

As the designer's additions say: no generic "farmland". Each crop, tree and animal has a suitability profile over the land numbers (e.g. olive: no hard frost, deep well-drained soil, tolerates drought and slope; date: heat plus a reachable water table even under a desert sky; emmer tolerates poorer soil than bread wheat). A node's yield for a crop = best-case yield x suitability x the share of land worked, lowered by nutrient depletion, raised by manure, fallow, irrigation and tools. Herds eat forage, and overgrazing lowers next year's capacity; dung goes to fields *or* to the fire, a real choice.

CHANGE OVER TIME: FOUR SPEEDS

The designer's three, plus one:

- **Fixed** on game timescales: bedrock, slope, ore body locations, the shape of a natural harbour.
- **Renewable but vulnerable**: soil nutrients, standing timber, forage, fish stocks, groundwater. Each has a stock and a regrowth rate, and can be run down.
- **Seasonal**: rainfall, river level, soil moisture, grass, navigability. Suggestion: with years (or more) per turn, seasons aren't simulated one by one; each year draws its weather from the node's climate (drought frequency, flood risk), so harvests vary and a run of bad years is possible. Sailing seasons become a cap on how much sea trade a year can carry.
- **Slowly altered by people** (added): deforestation (Lebanon's cedars, the Greek hills), salinisation from irrigation (southern Mesopotamia), harbours silting up (Ephesus, Miletus, Ostia), marshes drained or spreading. These change the "fixed" layer over centuries.

HISTORY AS THE GUIDE, HERE TOO

Like population's historical ceiling, bots' land and extraction should follow history: Laurion's silver fading, Spanish mines booming under Rome, Mesopotamia's soils salting, forests shrinking where they did. The player alone can do better (or worse) than history. **(OPEN)**

GOODS LIST: ADDITIONS AND GROUPING

Suggested additions, all historically important on this map in 300 BC: sesame and sesame oil (Mesopotamia's oil, not olive), fodder crops (vetch, lucerne), cotton (a little, Egypt and beyond), natron (Egypt: glass, washing, embalming), bitumen (Dead Sea and Mesopotamia: caulking, mortar), alum (dyeing), cinnabar (red pigment, mercury), silphium (Cyrene's famous and later extinct export: a perfect vulnerable resource), cedar as its own ship and temple timber, garum (fish sauce), leather, war elephants (North African and Syrian herds, captured, not bred), ivory as a raw good, and goods from beyond the map edge (amber, silk, pepper and spices, Indian steel) arriving only by trade.

Suggestion: keep the designer's full list as the *goods*, but show and simulate them through ~12 **families** (the designer's own headings: field crops, orchards, animals, fish and gathering, grain products, oil and drink, fibre and cloth, wood and fuel, stone and earth, metals, metalwork, construction, prestige). The engine tracks every good; the screens lead with families and open to detail. Timber should have a few grades (fuel, building, long straight ship timber), since the designer notes that woodland good for fuel isn't ship timber.

Labour: in antiquity much work was done by enslaved people. Suggestion: model labour as a population input (free, dependent, enslaved), not as a tradeable good, so the history is represented without the game treating people as a commodity. **(OPEN)**

WHAT IT FEEDS

- **Carrying capacity:** the growth drivers' stand-in (3x historical population) becomes the food a region's land can grow: calories from crops, herds and fish at its current tech. This is the first link to build.
- **Growth factors:** food security (harvest against need), land and living standards (land per person), disease (marshes, malaria zones).
- **Attraction:** mines, ports, markets and quarries pull people.
- **Economy, military and Might** (later build steps): output, what can be built, which units can be raised.

SUGGESTED BUILD ORDER WITHIN STEP 2

1. The land layer: bake the six parts per node from real data, show each as a map view, and check it against known history (fields where HYDE has cropland, forests where KK10 has them).
2. Raw-resource potentials from the suitability profiles, and carrying capacity from food, replacing the 3x stand-in. Re-run the population balance tests.
3. Renewable stocks (soil nutrients, forests, forage, fish) with depletion and regrowth, and the yearly weather draw.
4. Goods and recipes, with the economy (build step 4), where they're first needed.

Each part gets balance tests like population's: a realm doing nothing tracks history, maxed effort stays within bounds, a depleted stock can recover.

DYNASTIES AND CHARACTERS (DECIDED, BUILT)

IN SHORT

Ruling families live, marry, have children and die. The crown passes by real inheritance rules, and a new house takes over when a line dies out or a throne is seized. Built and in the game.

Depth for now: enough for ruling families to live, marry, have children and succeed each other for the whole game, without traits, events or intrigue (those can be layered on later). Code: `src/Dynasties/` (`CharacterRegistry.cs` for succession and mortality, `CharacterRegistry.Life.cs` for the yearly simulation, `Names.cs`), the 300 BC families in `MapView.RealCivs` / `FrontierZones`. Tests: `DynastyTest`, `DynastyLifeTest`, `TimeTest`.

- **Starting families.** Each realm starts with its real (or, where the record is thin, plausible) 300 BC ruling family with real ages: Antigonos the One-Eyed at 82 with his son Demetrios and grandson Antigonos Gonatas, Ptolemaios I at 67 with Berenike and their children, Seleukos with Antiochos, and so on. Rome, a republic, is represented by its leading house (the Valerii). Rulers range from 38 to 82.
- **Cultures.** Every realm and character has one of eight naming traditions (Latin, Punic, Greek, Meroitic, Nabataean, Iberian, Celtic, Scythian). Children take the culture of the parent whose house they belong to; a first son is sometimes named for his paternal grandfather and a first daughter for her maternal grandmother.
- **Mortality.** A pre-modern elite life table: 8% die in the first year, about one in six before five, 0.8% a year from 15 to 39, then a rise from 1% at 40 doubling about every nine years (cap 35%). About 63% of 40-year-olds reach 60 and a third reach 70. Average reign comes out around 26 years.
- **The court.** Only each realm's court marries and has children: the ruler, spouse, children and grandchildren, siblings with their children, and the heir's household. Everyone else lives out their life but the family tree stops branching there, which keeps the world to a few hundred living people however long the game runs.
- **Marriage.** Unmarried court members (men 17-55, women 15-38) marry with a 35% chance a year; three in ten matches are with another realm's court (the closest in age within 15 years, from another house), the rest with a generated noble of the realm's culture.
- **Births.** A married woman of 16-42 has a child with a chance of 24% a year in her twenties, falling to 3% after 40; at most nine children. About four or five births per full marriage.
- **Succession.** Primogeniture with representation: the ruler's descendants in order, the eldest line first, so a dead eldest son's children come before his younger brother (under male preference, sons' lines before daughters' at every level). With no descendants, the search moves up: brothers and nephews, then uncles and cousins, up to four generations. A princess who married outside the great houses keeps her children in her house and line. **No personal unions:** someone already reigning elsewhere is passed over, so realms never merge by inheritance (a later decision could allow it).

- **New houses.** A realm gets a new ruling house when its family has died out, or when a strongman seizes the throne at a succession: 6% at every succession, plus 25% when the heir is a child under 16, plus 10% when the heir isn't the late ruler's own descendant. The founder (30-50, of the realm's culture) comes with a wife and children. Over two thousand years, houses last about 280 years on average.
- **The chronicle** shows successions and new houses in every realm, and births, marriages and deaths only in the player's own court.
- **Known limits.** Dead characters are kept for family history, so the save grows with time (about 7 MB of family records by AD 1700). The HUD shows the heir's name but not why they're heir (a family tree view is future work).

NEW CONCEPTS NEEDED

IN SHORT

A shopping list of things later systems will need: named settlements, capitals, resources, roads, Might, map overlays and the conquest panel.

- **Settlement** — named populated place: id, name, position, population, tier, founded year, plus a status (**active** / **ruin**) for the hysteresis and ruin rules in Population & settlement engine. Tracked separately from provinces: a province can contain several, and unclaimed or unorganized land carries whichever clusters clear the percentile threshold. Named status survives conquest and loss of ownership.
- **Capitals** — each realm has one country capital, and each organized province has one province capital (its administrative seat). Both are a settlement reference plus a "first designated" record per realm, so the one-time resettlement can't repeat. They drive the growth bonuses in Capital growth bonuses.
- **Resource categories** — coal, ores (iron/copper/tin/gold/silver/lead), water, flora/fauna (timber/game/fish/wild plants), plus categories that tie into the existing civ flavor text rather than inventing a separate trade- goods system later: Kush's iron industry, Nabatea's incense routes.
(OPEN) full list not finalized.
- **Infrastructure tier per cell** — none/dirt/gravel/paved/rail/electrified, hard-capped at gravel while unorganized.
- **Might** — see the dedicated section below. Supersedes the standalone "local militia" idea by generalizing it into the domain-split strength score used everywhere annexation resolves, for both realms and raw land.
- **Map overlays** (UI) — a control to switch what the map draws on top of the terrain: Political (today's ownership view, the default), **Regions** (the ancient geographic regions and their names, the only place those names appear; built as a toggle), and later the population heatmap and resource overlays. One overlay active at a time.
- **Floating confirmation panel** (UI) — live Might + resource-icon readout while painting; see Annexation UX, below.

THE FIVE SIMULATION SYSTEMS (REALM/PROVINCE LEVEL)

IN SHORT

The five big systems a realm runs on: population, resources, economy, technology and military.

- **Population** (per province, summed from the density field) — how many people there are comes from natural increase (food security, disease, urban penalty, security, land and living standards, burden, health, climate), and where they live from attraction and bounded migration; see Growth drivers. Carries integration/sentiment (freshly conquered = low, decays toward assimilated over years unless suppressed by unrest).
- **Resources** (per province, summed from the density field) — yield type from terrain; gates military unit types and economic output.
- **Economy** (realm-level, aggregated from provinces) — treasury, income (tax from population x output) minus expenses (upkeep, admin cost).
- **Technology** (realm-level) — military / administrative / agriculture tracks; administrative tech is what should eventually cap how much territory a realm can hold before unrest/corruption penalties bite.
- **Military** (realm-level armies drawn from province manpower + resources) — recruited units gated by resources and tech; a leader/general slot extends the existing `Character.traits`, not a new character concept.

MIGHT: THE UNIFIED STRENGTH SCORE

IN SHORT

One strength score per realm or piece of land, split into Land and Naval (Air and Space later), compared whenever someone tries to take land. Unowned land defends with its local population.

The number actually compared whenever annexation resolves. Split into domains that unlock by era: **Land** and **Naval** (both available from the game's 300 BC start — ancient/medieval navies were real, triremes and quinqueremes included, and several starting civs are explicitly maritime powers: Carthage, Rome, the Greek world, Ptolemaic Egypt), **Aerial** (~1900 AD), and eventually **Space**. A domain that hasn't unlocked yet shows as o/greyed out for every realm equally. The data model carries all four slots from day one (cheap now, expensive to retrofit later) even though two sit inert for a long time. **(DECIDED)**

(The ~1500 AD instinct isn't wrong, just describes a different thing — true ocean-crossing "Age of Sail" capability, as opposed to Naval Might existing at all. That's a plausible later refinement, e.g. a coastal/riverine vs. blue-water distinction, not something blocking Naval Might being active from turn one.)

Where a Might number comes from depends on what's being measured: - **A realm's own Might** (the attacker, or an organized province's defender) — derived from that realm's Military system (armies, tech, fortification), per domain. - **Unclaimed or unorganized land's Might** (defender only — raw land has no attacker side) — derived directly from local population density: the same per-cell density field already planned under Layers, read live, not a separately baked asset. "Heat map" here describes the *display* (a colored intensity overlay), not a second data source that could drift out of sync with the density field it's reading.

This resolves both open questions from before: - Defense is **not** seat-anchored — it's the whole province's aggregate Might, so beating it (in whichever domain(s) apply — Land and Naval, for a very long time) flips the entire province regardless of exactly which cells got painted, matching the earlier "conquered a state from one city" decision. - Resolution reads as **instant at confirm-time** — one decisive Might comparison when you commit, not a multi-tick siege bar. (Inferred from "confirm... if you can handle it" rather than stated outright — flagging in case a multi-year siege was actually intended.)

One refinement folded back into Layers: since Might for unclaimed land is explicitly meant to reflect *historical* population, the density field itself is sourced from **HYDE** (History Database of the Global Environment — PBL Netherlands Environmental Assessment Agency / Utrecht University), which gives gridded population estimates from 10,000 BCE to the present (HYDE 3.2.1, CC BY 3.0). Free, public, no licensing cost — same footing as the Natural Earth coastline and historical-basemaps borders already in the project. Implementation follows the same pattern as `tools/build_land_mask.py`: a new `tools/build_population_mask.py`-style import step bakes HYDE's grid into textures on this project's own projection. One snapshot is baked per HYDE keyframe the game's timeline covers, starting at 300

BC, since the snapshots double as the **H_hist** keyframes in Population & settlement engine. Terrain-driven estimation is dropped as the source of record; it remains only as a plausible gap-filler if HYDE's resolution turns out too coarse for a specific region. **(DECIDED)**

ANNEXATION UX: FOG OF WAR AND THE FLOATING CONFIRMATION PANEL

IN SHORT

You can't see a target's strength until you start painting over it. Then a floating panel shows the strength and resources of exactly what you've painted, province by province, before you confirm.

A target's Might is hidden until you start painting over it — no defense numbers just from looking at the map. Starting a paint stroke on unclaimed or enemy territory triggers the reveal, and it updates live as the stroke grows or shrinks, reusing the same dirty-cell tracking the ownership proposal paint already has.

While painting, a floating confirmation panel shows, live: - Might per domain for whatever's currently painted (Land and Naval as real numbers; Aerial/Space greyed out at 0 until their era unlocks) - Icons for whichever resource types are present in the currently-painted cells

Confirm stays available regardless of whether you're favored — showing the numbers is about an informed choice, not a blocked action. Losing (per the existing rule below) means the paint snaps back with attrition to the attacker's own Might, not that confirm gets disabled. **(DECIDED)**

When one stroke spans multiple distinct provinces (or crosses from unclaimed land into an organized enemy province), the panel shows a **breakdown per province** — a separate Might/resource readout for each one touched, not a blended total. Matches how resolution itself happens: each affected province is checked against its own Might independently, so the panel should show exactly what's about to be decided, not an average that hides which individual provinces you can and can't take. **(DECIDED)**

ANNEXATION RESOLUTION

IN SHORT

Conquest stays a brush stroke. On confirm, each province you painted is compared against your Might: win and you take the whole province, lose and the paint snaps back with losses to your army.

Stays a paint action, not a menu. But the *unit of capture is the province*, not the cell, once land is organized: painting/confirming inside an enemy province checks that province's Might (see above — garrison/fortification, derived from the owning realm's military + tech + that province's integration) against the attacker's Might. Beating it flips the *entire* province — every member cell, including ones the brush stroke never touched, keeps its existing `province_id`, population, infrastructure, and settlements, just under a new owning realm (mechanically: the province's owner pointer flips, then a bulk write propagates that to its cells, per the invariant above). Same brush, same immediate feedback — the simulation decides whether paint sticks, at province granularity, not a separate siege UI. (DECIDED)

Unorganized land (enemy-held, but never administered into a province) has no "whole shape" to inherit, so it still resolves locally — per painted cell or contiguous painted blob — against its population-scaled militia. Unclaimed land (no owner at all) resolves the same way, against its own native militia — usually weak given how sparse unclaimed populations tend to be, but not an unconditional freebie either.

This also settles the earlier open question about post-conquest organization: captured organized provinces do **not** reset to unorganized and don't need a separate "decide how to organize this" step. They arrive already organized, under new management, exactly as they were run before.

Since a province is now captured as an atomic whole, **province size is a real balancing lever**: small/numerous provinces make conquest gradual, a few huge provinces make it all-or-nothing and swingy. Worth weighing during Phase 1 (province generation/seeding), not just an implementation detail.

SUGGESTED BUILD ORDER

IN SHORT

The order the systems are built in, each checked in the real game before the next begins. Step 1 (provinces) is done; step 2 (population) is partly done.

1. Province layer (`province_id` grid + brush-editable boundaries), no stats yet — verify rendering/paint behavior only.
2. Density fields (population + resource-type per cell), provinces sum on demand. Population via the engine: HYDE-sourced `H_hist`, `H_sim` seeded from it, blend, then $C * H^k$. Start with `H_sim` static and verify the 300 BC output against HYDE before adding any dynamics. Then the historical ceiling, with a player/non-player split, and capital bonuses; rerun `tools/calibrate_population.py` on the real grid. Then the Growth drivers groundwork: regional totals, conserving migration, and the balance-test harness, before any later system writes a factor score.
3. Unorganized-territory capability gating (the matrix above) + settlements (clustering, percentile naming, hysteresis, ruins), plus the regional floor. (The region mask and Regions overlay were built with the growth drivers, which needed them.)
4. Economy (realm treasury from province sums).
5. Military + tech (army/power score, minimal tech multipliers).
6. Integration/sentiment (decay-toward-assimilated stat).
7. Paint-gated annexation (the resolution function tying 2-6 together).

Status: population engine and growth-driver groundwork built. Built so far:

`src/World/PopulationEngine.cs` (stages 1-2, the historical ceiling with player legacy fade, capital bonuses and resettlement, HYDE-based seeding and 25-year natural resettlement, save/load), the growth drivers' two channels with every factor at its baseline (`PopulationEngine.Growth.cs`), the 38 geographic regions (`tools/build_region_mask.py`, `PopulationEngine.Regions.cs`) and the Regions overlay (`MapView.Regions.cs`), `tools/build_population_mask.py` (the HYDE import), the engine wired into world generation, the yearly tick, saves, and a HUD population line, and each real civ's 300 BC capital registered as an existing capital. Covered by `src/Tests/PopulationEngineTest.cs` (toy-world mechanics), `src/Tests/PopulationHydeTest.cs` (the calibration on the real grid) and `src/Tests/PopulationBalanceTest.cs` (the growth drivers' balance tests). Not built yet: stage 3 (clusters, percentile naming, ruins' labels, the regional floor), gameplay systems that write factor scores and attraction (the engine exposes `SetFactor` and `DriverMods` for them; until they exist the world follows history), events that trigger `ApplyShock`, and province capitals. The HYDE 3.2.1 keyframes are baked into `data/population/` (300 BC to 2017 AD, the 300 BC one derived; see `MAP_DATA.md`), so the engine runs on the real grid: 284,626 land nodes, loaded in ~0.5 s. The whole game is C# (ported from

GDScript with identical results): a yearly population tick takes ~30 ms and generating the world ~3 s. The old known issue (the non-player world lagging HYDE's total as history rose) is gone: regional totals now follow HYDE exactly.

Each phase should be independently verified against the real headless Godot engine (the Godot 4.7.2 .NET build; the session hook installs it) before moving to the next. The tests are under `src/Tests/`, one scene-tree script per suite, run with `godot --headless --path . --script res://src/Tests/<Name>.cs`; each exits with 1 if any check fails.